

MAESTRÍA EN INTELIGENCIA ARTIFICIAL APLICADA

PROYECTO INTEGRADOR

Avance 4. Modelos alternativos

EQUIPO 52

A01795991 - José Antonio Fernández Pérez

A01659811 - Juan José Ponce

A00378771 - Hiram García Austria

ASESOR

Dr. Gerardo Jesús Camacho González

25 de Octubre de 2026

Índice

Modelos 3

Ajuste de parámetros 9

Comparativa de modelos 13

Matriz de confusión 21

Conclusión 27

Referencias 28

Modelos

En este avance evaluamos seis arquitecturas de aprendizaje automático para predecir el siguiente evento de un partido de fútbol y, cuando corresponde a un pase, identificar al posible receptor del balón.

Las arquitecturas estudiadas son Gradient Boosting, GraphSAGE, GATv2 relacional, GNN relacional con GRU temporal, Graph Transformer espacial y Graph Transformer espacial-temporal. Adicionalmente, evaluamos una variante del último modelo que incorpora ponderación de clases para mejorar el reconocimiento de eventos poco frecuentes.

Los modelos representan diferentes niveles de complejidad, desde métodos tabulares tradicionales hasta redes neuronales capaces de modelar relaciones espaciales y dependencias temporales entre jugadores.

El protocolo y las características parten del repositorio desarrollado por el equipo (García Austria, s. f.). Los resultados numéricos corresponden a las ejecuciones descritas en este informe.

Objetivo

El objetivo es predecir el siguiente evento y, cuando corresponde a un pase, su receptor, a partir del estado observado de jugadores y balón. Las etiquetas de entrenamiento son referencias heurísticas extraídas del tracking, no eventos oficiales verificados. Además de evaluar la predicción, exploramos representaciones vectoriales aprendidas de estados de juego.

Condiciones generales

1. Solo se predice cuando el balón está en juego y un jugador lo tiene: el más cercano del equipo con la posesión, a 1.5 m o menos del balón.
2. Se toma una muestra 10 veces por segundo.
3. La respuesta correcta es la primera jugada que ocurre en los siguientes 3 segundos. Si no ocurre ninguna, es "mantener el balón".
4. Un toque de menos de 0.2 s entre dos toques del mismo jugador o equipo se ignora, porque es ruido del seguimiento.
5. Las jugadas a balón parado (gol, tiro de esquina, saque de puerta, de banda, fuera de juego, falta) se reconocen por dónde se reanuda el juego, y cuentan como ocurridas en el último instante con el balón en juego.

Jugadas y condiciones

Jugada	Condición para contarla
Pase	El balón pasa a un compañero, recorre al menos 3 m y viaja como máximo 4 s.
Pérdida	El balón pasa a un rival sin salir del juego.
Tiro	Medio segundo después del toque, el balón va a 12 m/s o más. El tirador está a 35 m o menos de la portería, la trayectoria cruza la línea de gol entre los postes, con un margen de 6 m, y ningún compañero lo recibe.
Gol	El juego se reanuda desde el centro del campo, a 2 m o menos del punto central. El último tiro del equipo que anotó en los 10 s anteriores se convierte en el gol.
Tiro de esquina	El juego se reanuda a 3 m o menos de una esquina.
Saque de puerta	El juego se reanuda dentro del área chica, con un margen de 1 m.
Saque de banda	El juego se reanuda a 1.5 m o menos de una línea de banda.
Fuera de juego* (aproximado)	El juego se reanuda en otro lugar, y en los 6 s anteriores hubo un pase a un atacante más de 0.5 m adelantado al penúltimo defensor, que está a 8 m o menos del balón cuando se detiene el juego.
Falta* (aproximado)	El juego se reanuda en el punto de penal, o en cualquier otro lugar que no encaje con lo anterior. Por eso también incluye lesiones, revisiones del VAR, etc.
Posesión	Nada de lo anterior ocurre en los siguientes 3 segundos.

*Fuera de juego y falta son las menos fiables. Hacen falta los datos oficiales de eventos (matchFeed.json) para tener etiquetas exactas.

Árboles de decisión con Gradient Boosting

Es el modelo de referencia (baseline) ya que es un clasificador de tipo Gradient Boosting basado en histogramas, implementado mediante “HistGradientBoostingClassifier” de scikit-learn, entrenado con características diseñadas manualmente.
Su configuración consta en realidad de dos modelos:
Modelo de evento: un clasificador multiclase que predice 10 posibles resultados, tomando una fila por frame (fotograma) con 35 características:

- 17 características del nodo portador del balón (posición, velocidad, dirección, rol, presión, distancia y ángulo respecto a la portería, etc.);

- 8 características globales (posición del balón, altura, velocidad, periodo, tiempo, número de jugadores por equipo);
- 10 características diseñadas manualmente: distancia al defensor más cercano, número de defensores en un radio de 5 m y 10 m, distancia al compañero más cercano, número de atacantes y defensores por delante del balón, línea de fuera de juego (segundo defensor más retrasado), posición media en el eje X de cada equipo y número de líneas de pase abiertas.

Modelo de receptor: un clasificador binario que para cada par (frame, compañero candidato), responde a la pregunta: ¿Es este compañero el receptor?. Las puntuaciones de los candidatos dentro de un mismo frame se ordenan.
La implementación se basa en árboles potenciados por gradiente con discretización por histogramas, conforme a la documentación vigente de HistGradientBoostingClassifier (scikit-learn developers, s. f.). Utilizamos dos clasificadores: uno para el evento y otro para puntuar candidatos a receptor.

¿Por qué este tipo de modelo?

1. Es una referencia adecuada para las características tabulares de este problema. Permite modelar relaciones no lineales e interacciones entre variables sin construir una red de relaciones explícita. Su elección no implica superioridad universal frente a redes neuronales; esa comparación debe establecerse con los resultados del experimento.
Esta referencia permite evaluar cuánto aportan las representaciones de grafos frente a características tabulares diseñadas manualmente. Su costo de entrenamiento y su capacidad para capturar interacciones no lineales la hacen apropiada para establecer un punto de comparación; no se presupone que supere a las redes neuronales.

GraphSAGE

GraphSAGE (Graph Sample and Aggregate) es un marco de redes neuronales de grafos (GNN) diseñado para generar representaciones vectoriales (embeddings) de nodos en grafos muy grandes de manera inductiva. El aprendizaje inductivo mediante agregación de vecinos se fundamenta en Hamilton et al. (2017).
Maneja grafos de formas variables, el número de jugadores puede cambiar (debido a tarjetas rojas o a la pérdida de datos de seguimiento); el modelo aprende una función de agregación que funciona para cualquier conjunto de vecinos, por lo que el grafo no requiere un tamaño fijo.
También admite distintos tipos de relaciones. Una versión heterogénea permite asignar pesos diferentes a los compañeros de equipo y a los oponentes. Tener un compañero cerca implica una opción de pase, mientras que tener un oponente cerca implica presión.
Un fotograma de un partido de fútbol puede representarse, de forma natural, como un grafo: 22 jugadores (nodos) junto con las relaciones entre ellos (aristas). Lo que sucede a continuación depende menos de la posición individual de un jugador que de cómo interactúan los jugadores entre sí: quién está libre, quién marca a quién y qué líneas de pase están abiertas. Una red neuronal de grafos (GNN, por sus siglas en inglés) aprende

estas relaciones directamente, en lugar de depender de resúmenes definidos manualmente, como ocurre en el modelo base de gradient boosting.

Configuración

El grafo: Uno por fotograma.
Nodos: los 22 jugadores, situados en posiciones fijas. Los nodos 0–10 corresponden al equipo en posesión del balón, y los nodos 11–21, al equipo rival. Las coordenadas se ajustan de modo que el equipo en posesión siempre ataque en la dirección positiva del eje x (+x).
Características de los nodos: posición, velocidad, rapidez, dirección de movimiento, rol (portero, defensa, centrocampista, delantero), distancia al balón, si el jugador tiene el balón, equipo en posesión, presión (oponentes a menos de 3 m), y distancia y ángulo respecto a la portería.
Aristas: reconstruidas a partir de las posiciones en cada lote (batch):

- Compañero de equipo: cada jugador se conecta con sus 5 compañeros más cercanos; el jugador que tiene el balón se conecta con todos sus compañeros (representando cada opción de pase).
- Oponente: cada jugador se conecta con sus 5 oponentes más cercanos.
- Características globales (8): posición, altura y velocidad del balón, periodo de juego, tiempo transcurrido en el periodo y número de jugadores por equipo.

El codificador utiliza una proyección de entrada y tres capas de agregación relacional inspiradas en GraphSAGE. Cada jugador combina su estado propio con resúmenes separados de compañeros y rivales, mediante max-pooling aprendido, conexiones residuales, dropout y normalización de capa. La implementación propia opera sobre máscaras densas de 22 por 22 nodos.

Dos cabezales de salida:

1. Cabezal de eventos: combina el embedding del jugador que tiene el balón, los valores promedio y máximo de todos los jugadores, y las características globales; genera probabilidades para 10 posibles desenlaces (pase, pérdida de balón, disparo, gol, saque de esquina, saque de puerta, saque de banda, fuera de juego, falta, retención del balón) en los siguientes 3 segundos.
2. Cabezal de receptor: asigna una puntuación a cada compañero de equipo basándose en los embeddings del jugador que tiene el balón y del propio compañero, además de características del par (distancia, diferencias en las coordenadas x/y, y si la línea de pase está despejada). Una función *softmax* aplicada a estas puntuaciones determina el receptor más probable.

La diferenciación de compañeros y rivales utiliza tipos de relación, aunque todos los nodos representen jugadores. Esta formulación es coherente con el aprendizaje en grafos heterogéneos por tipos de arista descrito en la documentación vigente de PyTorch Geometric (PyTorch Geometric contributors, s. f.); la implementación del equipo usa PyTorch y máscaras densas, sin depender de esa biblioteca.

GATv2 relacional

Incorporamos una red de atención de grafos con parámetros separados para relaciones entre compañeros y entre rivales. GATv2 permite que la atención dependa conjuntamente del nodo que recibe el mensaje y de su vecino; adoptamos esta idea para ponderar las interacciones relevantes del estado actual (Brody et al., 2022). La implementación es una adaptación relacional para grafos pequeños de fútbol.

Reutilizamos las 17 características por jugador, ocho características globales y cuatro características por candidato a receptor del pipeline original. El balón se representa mediante las variables globales, no como un nodo adicional. Las aristas son exactamente las construidas por build_relations: cinco compañeros y cinco rivales próximos, con conexiones entre el portador y todos sus compañeros.

Cada capa calcula atención independiente por relación y por cabeza. Los mensajes se combinan con el estado propio mediante una conexión residual, activación GELU, dropout y normalización de capa. Después se concatenan la representación del portador, el promedio y máximo de nodos válidos y las variables globales; una proyección produce un vector de 32 componentes.

Dos cabezales utilizan ese vector: uno genera diez puntuaciones de eventos y otro puntúa a los candidatos a receptor, junto con las representaciones de portador y compañero y sus características del par. Se excluyen el propio portador, los rivales y los jugadores ausentes. La probabilidad del receptor es condicional a que ocurra un pase; no es una probabilidad de pase conjunta.

Esta primera arquitectura representa un estado instantáneo enriquecido con variables de movimiento. No procesa una secuencia ni constituye por sí sola una representación de la jugada completa. Su vector puede servir de base para el modelo temporal posterior.

GNN relacional con GRU temporal

Este modelo incorpora una secuencia de estados del juego, en lugar de utilizar únicamente el fotograma actual. La GRU procesa observaciones ordenadas y genera una representación vectorial del fragmento reciente. Las unidades recurrentes con compuertas permiten aprender qué información conservar a lo largo de una secuencia (Cho et al., 2014).

Implementamos un entrenamiento en dos etapas. Primero reutilizamos el GATv2 relacional entrenado en la partición 42 y congelamos sus 27,403 parámetros. Este codificador produce un vector espacial de 32 componentes por observación y representaciones de los jugadores. Después entrenamos una GRU unidireccional y nuevos cabezales de evento y receptor. La red de grafos de esta ejecución es el GATv2 relacional descrito antes; no se entrenó una nueva GNN desde cero ni se optimizó todo el sistema conjuntamente.

La GRU recibe hasta 20 vectores históricos y el intervalo temporal entre observaciones. Su estado oculto tiene 64 componentes. La proyección final combina esa memoria con el vector espacial del instante actual y produce una representación temporal de 64

componentes. El cabezal de receptor combina esta representación con las de portador y candidatos actuales y las cuatro características del par. Se mantienen las mismas diez clases y las mismas restricciones de receptores válidos.

La implementación recurrente utiliza torch.nn.GRU y secuencias empaquetadas para que el relleno no intervenga en el estado temporal, de acuerdo con la documentación vigente de PyTorch (PyTorch contributors, s. f.).

Construcción de las secuencias temporales

Conservamos la partición 42 y todos sus ejemplos: 901,687 de entrenamiento, 122,265 de validación y 118,786 de prueba. Para cada instante utilizamos solo observaciones con frame menor o igual al actual, del mismo partido y periodo. Reiniciamos la historia ante pausas detectadas en la serie original del balón, cambios de equipo en posesión o huecos mayores a 0.16 segundos. También limitamos la ventana a dos segundos y 20 observaciones. Estas reglas mantienen una orientación de ataque coherente dentro de cada historia.

Las historias son de longitud variable y su padding no interviene en la GRU. En prueba hubo 11.02 observaciones en promedio y 10 de mediana; el intervalo cubierto medio fue 0.982 s. El 26.28 % alcanzó las 20 observaciones y el 7.52 % solo tenía el instante actual. No eliminamos ejemplos por falta de una historia completa, lo que conserva la misma población de evaluación que los modelos anteriores.

Se verificaron los 1,142,738 ejemplos: ausencia de índices futuros y de cruces entre pausas, periodos o posesiones; ventana máxima de dos segundos; receptores válidos; e independencia del resultado respecto de los valores de padding. Estos controles se aplican a la construcción de las ventanas. La evaluación sigue siendo offline porque conserva las características cinemáticas centradas del pipeline original, limitación ya descrita para GATv2.

Graph Transformer espacial

Como quinta alternativa, implementamos un Graph Transformer espacial para analizar las relaciones entre los jugadores presentes en un instante del partido. A diferencia de GraphSAGE, que agrega información de vecinos mediante funciones de agregación, esta arquitectura utiliza mecanismos de atención multi-cabeza para aprender la importancia relativa de las interacciones entre jugadores.

Cada observación se representa mediante 22 nodos correspondientes a los jugadores, con 17 características individuales por nodo. Adicionalmente, se incorporan las posiciones de los jugadores, máscaras de validez, información del portador del balón, ocho características globales del estado del partido y cuatro características para cada candidato a receptor.

La arquitectura implementada contiene dos capas de atención espacial, cuatro cabezas de atención y una dimensión oculta de 64 componentes, con un total de 95,059 parámetros.

El modelo utiliza dos cabezales de predicción:

1. **Predicción de eventos:** clasifica el siguiente evento entre diez posibles categorías, incluyendo pases, pérdidas, tiros y situaciones de balón parado.
2. **Predicción del receptor:** estima cuál de los compañeros del equipo en posesión recibirá el balón cuando corresponda un pase.

Esta arquitectura permite modelar las relaciones espaciales entre jugadores sin depender exclusivamente de reglas de agregación previamente definidas.

Graph Transformer espacial-temporal

Como sexta alternativa, desarrollamos una arquitectura que incorpora información temporal al Graph Transformer espacial.

El objetivo es que el modelo no dependa únicamente de la configuración actual de los jugadores, sino que también pueda utilizar información de observaciones anteriores para identificar patrones de movimiento y cambios en el estado del partido.

La implementación se realizó mediante un entrenamiento en dos etapas. Primero, se utilizó el Graph Transformer espacial previamente entrenado como extractor de características. Posteriormente, se entrenó un Transformer temporal que procesa secuencias de hasta ocho observaciones consecutivas.

Para construir las secuencias se utilizaron los identificadores de fotograma y se limitaron los intervalos entre observaciones, evitando incorporar información de instantes posteriores al momento de predicción.

La arquitectura temporal cuenta con 103,381 parámetros y mantiene dos objetivos de predicción: el siguiente evento y el posible receptor del balón.

Este enfoque permite explorar si la incorporación de información histórica mejora la predicción respecto a un modelo que únicamente utiliza información espacial.

Variante con ponderación de clases (Modelo 6B)

Adicionalmente, se implementó una variante del modelo temporal que incorpora pesos por clase en la función de pérdida.

Esta modificación responde al considerable desbalance de las etiquetas de eventos: algunas categorías aparecen con mucha frecuencia, mientras que otras, como goles y determinadas jugadas a balón parado, tienen una representación muy reducida.

Los pesos se calcularon utilizando únicamente las observaciones de entrenamiento, asignando mayor importancia relativa a las categorías menos frecuentes.

La arquitectura del modelo 6B es la misma que la del modelo 6. Por tanto, se considera un experimento adicional de entrenamiento y no una séptima arquitectura independiente.

El objetivo de esta variante es evaluar si la ponderación permite mejorar el reconocimiento de eventos minoritarios, incluso cuando esto implique una reducción en la precisión global.

Ajuste de parámetros

La selección de parámetros se realizó utilizando los conjuntos de validación correspondientes a cada protocolo experimental. Las configuraciones de Gradient Boosting, GraphSAGE, GATv2 y GNN con GRU se desarrollaron utilizando el conjunto original de partidos, mientras que los Graph Transformers espacial y espacial-temporal se evaluaron mediante una selección reducida debido a las limitaciones computacionales y al tiempo disponible. No se realizó una búsqueda exhaustiva de hiperparámetros común a las seis arquitecturas.

Árboles de decisión con Gradient Boosting

El desarrollo original documentó el cambio de 100 a 300 iteraciones, tasa de aprendizaje de 0.5 a 0.1 y paciencia de 30 a 20 rondas. La reproducción empleó los valores finales: máximo 300 iteraciones, learning_rate=0.1, early_stopping=True y n_iter_no_change=20, con semilla 42 y ponderación inverse_sqrt_freq para el clasificador de eventos (García Austria, s. f.).
El clasificador original utiliza su mecanismo interno de parada temprana; no se sustituyó por una búsqueda con el conjunto externo de validación. La documentación de scikit-learn describe este comportamiento y sus parámetros (scikit-learn developers, s. f.). El clasificador binario de receptores se ajusta sobre compañeros candidatos de pases con receptor conocido.

GraphSAGE

Tuvo varias configuraciones, a continuación, se presentan únicamente las que cambiaron, elegidas por pérdida en validación.
Dimensión oculta de 128 a 64
Dropout de 0.1 a 0.3
Tasa de aprendizaje de 1e-3 a 3e-4
Weight decay de 0.01 a 0.05
Paciencia de early stopping de 5 épocas a 3 épocas

Parámetros específicos y compartidos

GraphSAGE utiliza tres capas, agregación max, cinco vecinos compañeros y cinco rivales, con batch de 256. Gradient Boosting utiliza iteraciones de árboles y no capas de grafos. Ambos aplican pesos de clase inverse_sqrt_freq.
La configuración por defecto se sobre ajustó a partir de la época 2 (validación 2.157).
La regularizada siguió mejorando hasta la época 9 (validación 2.142, test 0.804 frente a 0.813).

- Selección de la configuración de GraphSAGE: se entrenaron modelos tanto con la configuración predeterminada como con la definida en “regularized.yaml”, eligiendo la mejor opción según la pérdida (loss) en el conjunto de validación. El conjunto de prueba nunca se utilizó para tomar decisiones sobre la configuración. La configuración predeterminada (128 unidades ocultas) presentó sobreajuste a partir

de la segunda época (epoch), incluso con 46 partidos; por tanto, resultó mejor la versión regularizada (64 unidades ocultas y mayor tasa de dropout).

- El entrenamiento se realizó en una computadora local (no en la nube). GraphSAGE se ejecutó en una RX 9070 XT mediante la versión de PyTorch para AMD ROCm, con un tiempo de procesamiento de entre 70 y 80 segundos por época. El modelo base se ejecutó en la CPU.

GATv2 relacional

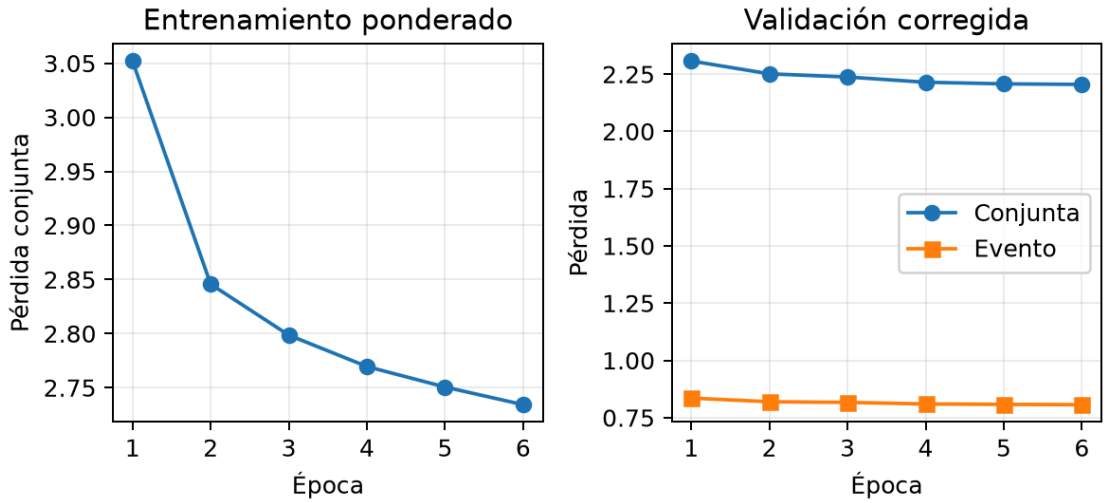
Configuración de GATv2	Valor
Capas relacionales	2
Dimensión de representación	32
Cabezas por relación	4
Parámetros entrenables	27,403
Dropout	0.30
Optimizador	AdamW
Tasa de aprendizaje	0.0003
Weight decay	0.05
Batch	256
Épocas máximas	6
Paciencia	3 evaluaciones
Semilla de la primera ejecución	42

La configuración compacta se eligió por el costo observado en CPU antes de evaluar prueba; no se presenta como una búsqueda exhaustiva de hiperparámetros. Se verificaron dimensiones de salida, gradientes finitos, máscaras e invariancia ante permutaciones coherentes de los nodos y relaciones.

La ejecución de GATv2 se realizó en CPU con cuatro hilos y PyTorch 2.14.1+cpu. Los tiempos por época se conservan en el historial; no son directamente comparables con los tiempos de GraphSAGE obtenidos en GPU.

Se ejecutaron 6 épocas y se seleccionó la época 6, con pérdida conjunta de validación 2.2049. Prueba se evaluó después de elegir el checkpoint. El presupuesto de seis épocas no demuestra convergencia ni optimización exhaustiva.

Evolución del entrenamiento y selección por validación



Nota. Elaboración propia; partición 42, 46 partidos de entrenamiento y seis de validación.

La pérdida de entrenamiento usa pesos de clase y aumento de datos; la de validación usa probabilidades corregidas. Por eso no deben restarse ambas curvas como si fueran métricas idénticas para diagnosticar sobreajuste. Se utiliza la evolución de validación para elegir el checkpoint.

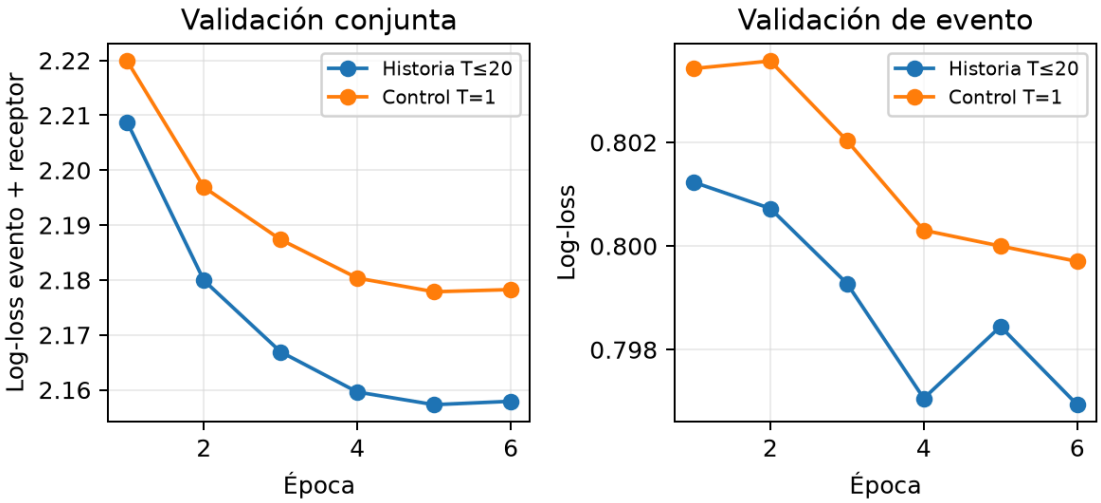
GNN relacional con GRU temporal

La segunda etapa utiliza 38,731 parámetros entrenables. El checkpoint GATv2 congelado contiene 27,403, incluidos sus cabezales originales, cuyas predicciones no se utilizan como salidas del modelo temporal. Ambos checkpoints almacenan 66,134 parámetros. Empleamos AdamW, tasa 0.0003, weight decay 0.05, dropout 0.30, batch 512, recorte de norma del gradiente a 1 y hasta seis épocas con paciencia de tres evaluaciones. No se aplica un reflejo por signo al vector latente; el codificador fue preentrenado con ese aumento sobre los datos físicos y después se mantuvo fijo.

El codificador congelado nunca se ajustó con partidos de prueba. La segunda etapa se seleccionó por la suma del log-loss de evento corregido por los pesos de clase y el log-loss de receptor en validación. El mejor checkpoint fue el de la época 5 de las seis ejecutadas. Se mantuvo la ponderación inverse_sqrt_freq de entrenamiento y el evaluador.

La secuencia se limita a 20 observaciones y dos segundos; la memoria oculta y la representación de salida tienen 64 componentes. El control T=1 mantiene codificador, cabezales, datos y presupuesto para aislar el aporte de la historia. En ambas variantes se seleccionó la época 5. Estas decisiones constituyen una configuración inicial y una ablación, no una búsqueda exhaustiva (Cho et al., 2014; PyTorch contributors, s. f.).

Selección por validación del modelo temporal y su control



Nota. Elaboración propia; entrenamiento y evaluación en partidos separados.

Graph Transformer espacial y temporal

Los modelos 5 y 6 se entrenaron en Google Colab utilizando una GPU NVIDIA T4 y PyTorch.

Debido al tiempo disponible para la experimentación, se utilizó una selección reducida de partidos, manteniendo una separación estricta entre entrenamiento, validación y prueba.

La distribución utilizada fue la siguiente:

- Entrenamiento: 10 partidos.
- Validación: 2 partidos.
- Prueba: 4 partidos.
- Semilla: 42.

Para el Graph Transformer espacial se utilizó una dimensión oculta de 64 componentes, cuatro cabezas de atención, dos capas espaciales, un tamaño de lote de 128 observaciones y tres épocas de entrenamiento.

El modelo temporal utilizó ventanas de hasta ocho observaciones y se entrenó durante tres épocas. Su arquitectura contiene 103,381 parámetros.

En ambos casos, los checkpoints se seleccionaron mediante el desempeño en validación y posteriormente se evaluaron sobre partidos reservados para prueba.

Durante el desarrollo del modelo temporal se detectaron valores infinitos negativos en las características utilizadas como entrada. Estos valores se trataron antes del entrenamiento para evitar pérdidas no finitas. También se incorporaron procedimientos de normalización y controles de estabilidad numérica.

Finalmente, se realizó un experimento adicional con ponderación de clases. Los pesos se calcularon a partir de la frecuencia de cada categoría en entrenamiento, con el propósito de reducir la influencia de las clases mayoritarias.

Estas configuraciones constituyen experimentos exploratorios y no representan una búsqueda exhaustiva de hiperparámetros.

Comparativa de modelos

Protocolos experimentales y alcance de las comparaciones

El conjunto de datos original comprende 58 partidos de fútbol, con aproximadamente 1.14 millones de muestras a una frecuencia de 10 Hz, considerando situaciones en las que el balón está en juego. Cada observación busca responder dos preguntas: dado el estado actual del partido, ¿qué evento ocurrirá en los próximos tres segundos? Y, si se trata de un pase, ¿qué jugador recibirá el balón?

Para evaluar las seis arquitecturas desarrolladas, se utilizaron dos protocolos experimentales, diferenciados principalmente por los recursos computacionales disponibles durante el desarrollo del proyecto. En ambos casos, la división de los datos se realizó por partidos y no por fotogramas, evitando que las observaciones de un mismo encuentro aparecieran simultáneamente en entrenamiento y prueba.

Protocolo 1. Gradient Boosting, GraphSAGE, GATv2 y GNN con GRU

Las primeras cuatro arquitecturas se evaluaron utilizando el conjunto original de 58 partidos, dividido en 46 partidos para entrenamiento, seis para validación y seis para prueba.

El experimento original de Gradient Boosting y GraphSAGE se repitió con tres particiones aleatorias por partido, correspondientes a las semillas 7, 42 y 123, con el propósito de examinar la sensibilidad de los resultados a la distribución de los datos. Por su parte, las nuevas ejecuciones de Gradient Boosting, GATv2 relacional y GNN con GRU temporal se compararon directamente utilizando la semilla 42.

Para garantizar la consistencia de los datos, reconstruimos las etiquetas y muestras mediante los módulos originales. La auditoría identificó 1,142,738 observaciones y confirmó que los conjuntos de prueba de las tres semillas suman 341,722 muestras, de acuerdo con el reporte original.

En particular, la partición correspondiente a la semilla 42 contiene 901,687 observaciones de entrenamiento, 122,265 de validación y 118,786 de prueba. Se utilizaron todos los ejemplos disponibles de entrenamiento para esta partición.

Los pesos de clase mediante el método *inverse_sqrt_freq* se calcularon exclusivamente con las observaciones de entrenamiento. En GATv2 se conservó el reflejo aleatorio de las variables físicas sobre el eje *y*, así como la función de pérdida conjunta para la predicción de eventos y receptores. En la etapa temporal con GRU no se aplicó dicho reflejo directamente sobre las representaciones vectoriales del codificador congelado.

La predicción del receptor se evaluó únicamente en pases con un destinatario identificado, independientemente de que el modelo hubiera clasificado correctamente el evento como

pase. La selección de las redes neuronales se realizó mediante la pérdida conjunta en validación, considerando el *log-loss* de eventos corregido por los pesos de clase y el *log-loss* del receptor. Esta corrección no garantiza por sí misma una calibración adecuada de las probabilidades, por lo que esta propiedad se evaluó mediante métricas específicas (Guo et al., 2017).

Protocolo 2. Graph Transformer espacial y espacial-temporal

Como ampliación del proyecto, desarrollamos dos arquitecturas adicionales: un Graph Transformer espacial y un Graph Transformer espacial-temporal. También evaluamos una variante del segundo modelo con ponderación de clases, denominada modelo 6B.

Debido a las limitaciones de recursos computacionales y al tiempo disponible para realizar los experimentos, fue necesario utilizar una selección reducida del conjunto original. En este caso, los datos se dividieron en diez partidos para entrenamiento, dos para validación y cuatro para prueba, manteniendo la separación por encuentros y utilizando la semilla 42.

El entrenamiento se realizó en Google Colab mediante PyTorch y una GPU NVIDIA T4. El Graph Transformer espacial utilizó dos capas de atención, cuatro cabezas de atención y una dimensión oculta de 64 componentes. Posteriormente, el modelo espacial-temporal incorporó secuencias de hasta ocho observaciones ordenadas cronológicamente, con el objetivo de aprovechar información histórica del movimiento de los jugadores.

Ambas arquitecturas se entrenaron durante tres épocas y sus checkpoints se seleccionaron utilizando el conjunto de validación. Finalmente, se evaluaron sobre los cuatro partidos reservados para prueba.

Adicionalmente, se entrenó una variante del modelo espacial-temporal con ponderación de clases. Los pesos se calcularon exclusivamente a partir de las frecuencias observadas en entrenamiento, con el propósito de mejorar el reconocimiento de eventos minoritarios. Esta variante conservó la misma arquitectura y partición de datos, permitiendo estudiar el efecto de modificar la función de pérdida.

Alcance de las comparaciones

Los resultados se interpretan considerando las diferencias entre ambos protocolos experimentales.

En el primer grupo, Gradient Boosting, GATv2 y GNN con GRU cuentan con evaluaciones directamente comparables en la semilla 42, mientras que los resultados originales de GraphSAGE y Gradient Boosting incluyen promedios de tres particiones. Por ello, el promedio de GraphSAGE se presenta como evidencia complementaria y no como una comparación emparejada con las nuevas arquitecturas.

En el segundo grupo, los modelos Graph Transformer espacial, espacial-temporal y espacial-temporal ponderado se evaluaron sobre las mismas 50,042 observaciones de

prueba. Esto permite comparar sus resultados dentro del protocolo reducido y analizar el efecto de incorporar información temporal y ponderación de clases.

Sin embargo, debido a que los Graph Transformers utilizaron menos partidos de entrenamiento y prueba que las primeras cuatro arquitecturas, sus resultados no deben interpretarse como evidencia de superioridad o inferioridad general frente a estas. Las diferencias observadas pueden estar relacionadas tanto con las arquitecturas como con el tamaño del conjunto de entrenamiento y las condiciones experimentales.

Por lo tanto, los experimentos con Graph Transformers constituyen una ampliación exploratoria del proyecto. Una comparación integral entre las seis arquitecturas requeriría entrenarlas y evaluarlas utilizando las mismas particiones, presupuestos de entrenamiento y métricas, preferentemente con múltiples semillas aleatorias.

Síntesis de los primeros cuatro modelos

Modelo	Evaluación	Log-loss evento ↓	Receptor top-1 ↑	Receptor top-3 ↑
Gradient Boosting	Semilla 42	0.8304	50.64 %	82.13 %
GraphSAGE	Promedio de tres semillas	0.8030	50.70 %	83.20 %
GATv2 relacional	Semilla 42	0.8202	49.89 %	82.89 %
GNN relacional + GRU	Semilla 42	0.8082	51.20 %	83.66 %

Nota. Elaboración propia con las ejecuciones del equipo y las cifras del reporte original. La fila de GraphSAGE es descriptiva y no establece un ranking emparejado entre los cuatro modelos. El promedio original de Gradient Boosting se conserva en la figura siguiente.

GraphSAGE frente a Gradient Boosting en tres particiones

En las tres particiones del experimento original, GraphSAGE mejora el log-loss de eventos y la predicción del receptor frente a Gradient Boosting por un margen pequeño. No domina todas las métricas: Gradient Boosting presenta menor ECE y mayor macro-F1 de clases fiables bajo la decisión argmax. Ambos tienen dificultades en resultados poco frecuentes.

Principales métricas (conjunto de prueba, promedio de las 3 divisiones)

Metric	Class prior	Nearest teammate	Gradient boosting	GraphSAGE
Event log-loss ↓	1.036	–	0.826 ± 0.011	**0.803 ± 0.013**
Event accuracy ↑	0.66	–	0.693	**0.699**
Calibration error (ECE) ↓	–	–	**0.010**	0.012
Macro-F1, reliable classes ↑	0.11	–	**0.261**	0.242
Macro-F1, rare classes favoured ↑	–	–	0.338	**0.341**
Receiver top-1 ↑	–	22.6%	50.0%	**50.7%**
Receiver top-3 ↑	–	~62%	81.8%	**83.2%**
Receiver log-loss ↓	–	–	1.443	**1.378**

↓ indica que un valor más bajo es mejor

↑ indica que un valor más alto es mejor

En cada división (split), la pérdida logarítmica (log-loss) de GraphSAGE para los eventos fue mejor por 0,025, 0,026 y 0,020, mientras que su métrica top-1 para el receptor superó a la referencia (modelo baseline) por entre 0,5 y 1,0 puntos. Esta consistencia es más importante que la magnitud de la diferencia.

¿Qué significa esto?

La pérdida logarítmica es la métrica principal. Mide cuánta probabilidad asigna el modelo a lo que realmente ocurrió. Realizar predicciones basándose únicamente en las frecuencias de las clases da una puntuación de 1,036.

En el promedio original, Gradient Boosting obtiene log-loss 0.826 y GraphSAGE 0.803. La reducción relativa frente a Gradient Boosting es de aproximadamente 2.8 %. Ambos mejoran la referencia por frecuencias de 1.036. El comportamiento con diez partidos se presenta como antecedente exploratorio, no como una comparación controlada del efecto del tamaño de datos.

El ECE global es bajo en la evaluación reportada, pero mide el acuerdo entre confianza de la clase elegida y frecuencia de aciertos. No garantiza que toda predicción de 70 % de pase corresponda a una frecuencia observada de 70 %. Para sostener esa afirmación se necesita calibración específica por clase y revisar los intervalos con suficiente soporte (Guo et al., 2017).

Predicción del receptor

En aproximadamente la mitad de los casos, el compañero al que el modelo asigna la mayor probabilidad es quien realmente recibe el balón; además, en el 83 % de las ocasiones, el receptor real se encuentra entre las 3 primeras opciones predichas por el modelo.

Elegir simplemente al compañero más cercano solo acierta el 23 % de las veces, lo que demuestra que el modelo está aprendiendo factores que van más allá de la simple distancia. Asimismo, una pérdida logarítmica más baja en la predicción del receptor implica que, incluso cuando el modelo se equivoca, asigna una probabilidad razonable al receptor real.

La precisión (accuracy) puede resultar engañosa en este contexto: dado que cerca del 66 % de las muestras corresponden a un "pase", predecir siempre "pase" arrojaría una precisión del 0,66. Por este motivo, la pérdida logarítmica y la métrica macro-F1 son indicadores más relevantes.

F1 por resultado (promedio calculado sobre las 3 divisiones)

Outcome	Gradient boosting	GraphSAGE
pass	0.81	0.82
loss	0.35	0.34
keep	0.37	0.41
shot	0.07	0.03
throw-in	0.08	0.03
corner kick	0.10	0.05
goal kick / goal / foul / offside	0.00–0.05	0.00–0.04

Resultados de jugadas comunes

Ambos modelos los manejan, y GraphSAGE es un poco mejor para diferenciar la jugada "mantener la pelota" a la de "pasar".

Resultados de jugadas poco comunes

Los tiros, las jugadas a balón parado, las faltas y los fueras de juego representan cada uno aproximadamente entre el 0,1% y el 2% de las muestras, y ambos modelos casi nunca los eligen como su respuesta principal, esto es el problema del desequilibrio de clases.
Con la decisión argmax, Gradient Boosting presenta mayor macro-F1 sobre las clases consideradas fiables: 0.261 frente a 0.242. El promedio se calcula sobre las clases presentes excluyendo gol, fuera de juego y falta; no es una métrica exclusiva de clases raras.

Al decidir con las probabilidades ponderadas antes de corregir los pesos, el macro-F1 fiable aumenta a 0.338 para Gradient Boosting y 0.341 para GraphSAGE. Esta es una regla de decisión distinta y se reporta por separado de la probabilidad corregida.
La partición 42 contiene solo cuatro muestras de fuera de juego en prueba, lo que impide estimar su desempeño de forma estable. El soporte corresponde a muestras, no a eventos independientes. Las faltas también presentan resultados muy bajos y su definición heurística agrupa situaciones diferentes; se requiere mejorar y validar esas etiquetas antes de atribuir el problema únicamente a escasez de datos.

GATv2 frente a Gradient Boosting en la semilla 42

Reentrenamos Gradient Boosting con su código original y los mismos partidos para establecer una comparación directa. Estos resultados no deben confundirse con el promedio de tres particiones de la tabla anterior.

Métrica	Gradient Boosting	GATv2 relacional
Log-loss de evento ↓	0.8304	0.8202
Accuracy de evento ↑	0.6921	0.6918
ECE global ↓	0.0117	0.0109
Macro-F1 fiable argmax ↑	0.2599	0.2077
Macro-F1 fiable ponderado ↑	0.3378	0.3274
Receptor top-1 ↑	0.5064	0.4989
Receptor top-3 ↑	0.8213	0.8289
Log-loss de receptor ↓	1.4173	1.4047

Las métricas de receptor utilizan 78,183 muestras de pase con destinatario identificado. Los pases cuya recepción no puede asociarse a un compañero válido se excluyen de esa evaluación, aunque permanecen en la evaluación del evento.

El log-loss de GATv2 fue 0.8202, frente a 0.8304 de Gradient Boosting. La reducción relativa fue 1.22 %; un valor negativo indica deterioro. La referencia de frecuencias obtuvo 1.0265. Para el receptor, elegir al compañero más cercano logró top-1 de 23.1 %, frente a 49.9 % de GATv2.

Respecto a Gradient Boosting, la diferencia de GATv2 en receptor fue de -0.75 puntos porcentuales en top-1 y +0.76 en top-3. Por ello, la evaluación de evento y la de receptor se interpretan como objetivos separados, sin resumirlas en una afirmación de superioridad general.

GNN relacional con GRU frente a los modelos de la semilla 42

Métrica	Gradient Boosting	GATv2	GNN + GRU
Log-loss de evento ↓	0.8304	0.8202	0.8082
Accuracy de evento ↑	0.6921	0.6918	0.6983
ECE global ↓	0.0117	0.0109	0.0156
Macro-F1 fiable argmax ↑	0.2599	0.2077	0.2320

Métrica	Gradient Boosting	GATv2	GNN + GRU
Macro-F1 fiable ponderado ↑	0.3378	0.3274	0.3304
Receptor top-1 ↑	0.5064	0.4989	0.5120
Receptor top-3 ↑	0.8213	0.8289	0.8366
Log-loss de receptor ↓	1.4173	1.4047	1.3681

El modelo temporal obtuvo log-loss 0.8082: una reducción relativa de 1.47 % frente a GATv2 y 2.67 % frente a Gradient Boosting. Su top-1 de receptor fue 51.20 % y su top-3, 83.66 %, evaluados sobre 78,183 muestras de pase con receptor identificado. La calibración global empeoró respecto a ambos comparadores y el macro-F1 fiable siguió por debajo de Gradient Boosting. La mejora no es uniforme entre objetivos.

Esta tabla reúne ejecuciones emparejadas de la semilla 42. No debe utilizarse para declarar superioridad frente al promedio de tres particiones de GraphSAGE, que se conserva como el experimento original. Tampoco se atribuye causalmente toda diferencia a la GRU: cambian la capacidad y el entrenamiento de los cabezales.

Control con un solo instante para valorar el aporte temporal

Entrenamos la misma GRU y los mismos cabezales con el mismo codificador congelado, semilla, pesos de clase, ejemplos, optimizador y presupuesto, pero limitando la entrada al instante actual. Este control T=1 es una ablación del modelo temporal, no otro algoritmo independiente para completar la rúbrica. Cada versión seleccionó su checkpoint exclusivamente con validación.

Métrica	Control T=1	Historia T≤20
Log-loss de evento ↓	0.8095	0.8082
Receptor top-1 ↑	0.5062	0.5120
Receptor top-3 ↑	0.8341	0.8366
Log-loss de receptor ↓	1.3824	1.3681

Frente al control, la historia redujo el log-loss de evento 0.16 % y aumentó el top-1 de receptor 0.58 puntos porcentuales. El aporte temporal es pequeño en esta ejecución. Una parte importante de la mejora frente a GATv2 ya aparece al cambiar y reentrenar los cabezales con T=1. No interpretamos esta única partición como prueba de significación estadística ni de comprensión táctica completa.

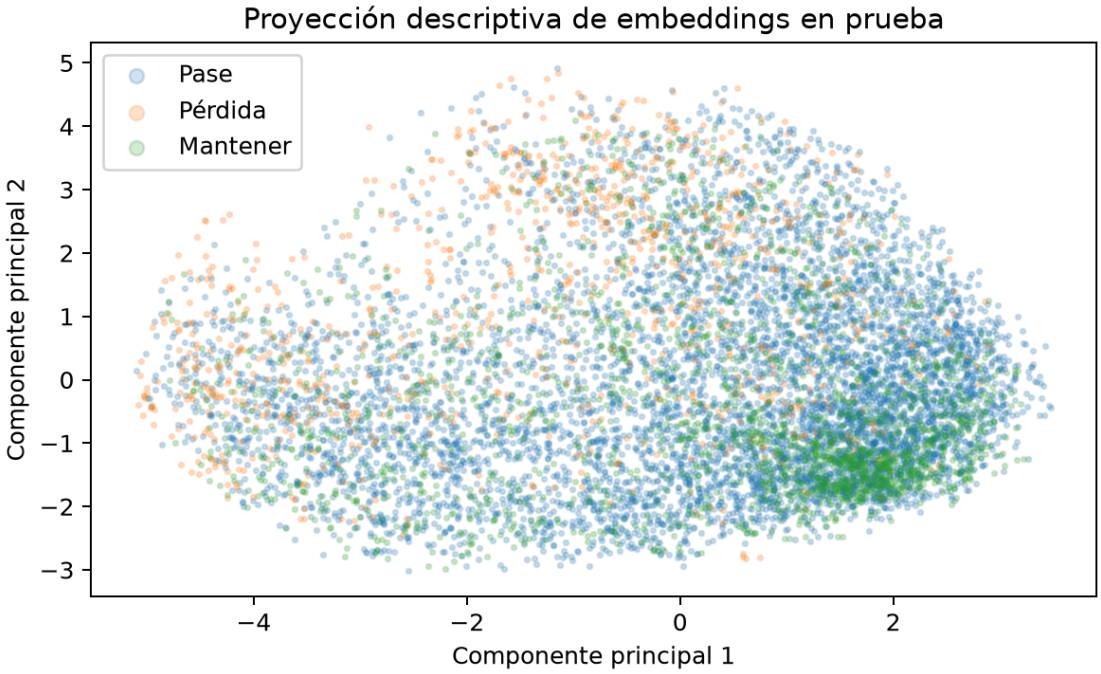
Representaciones aprendidas y calibración de GATv2

Evaluación de la representación aprendida

Congelamos el codificador y entrenamos una regresión logística sobre sus vectores de 32 componentes. La prueba lineal utilizó 30,000 ejemplos de entrenamiento, 10,000 de validación para elegir regularización entre C=0.1 y C=1, y 10,000 de prueba procedentes de partidos separados. Obtuvo log-loss 0.8300, accuracy 0.6854 y macro-F1 fiable 0.2147.

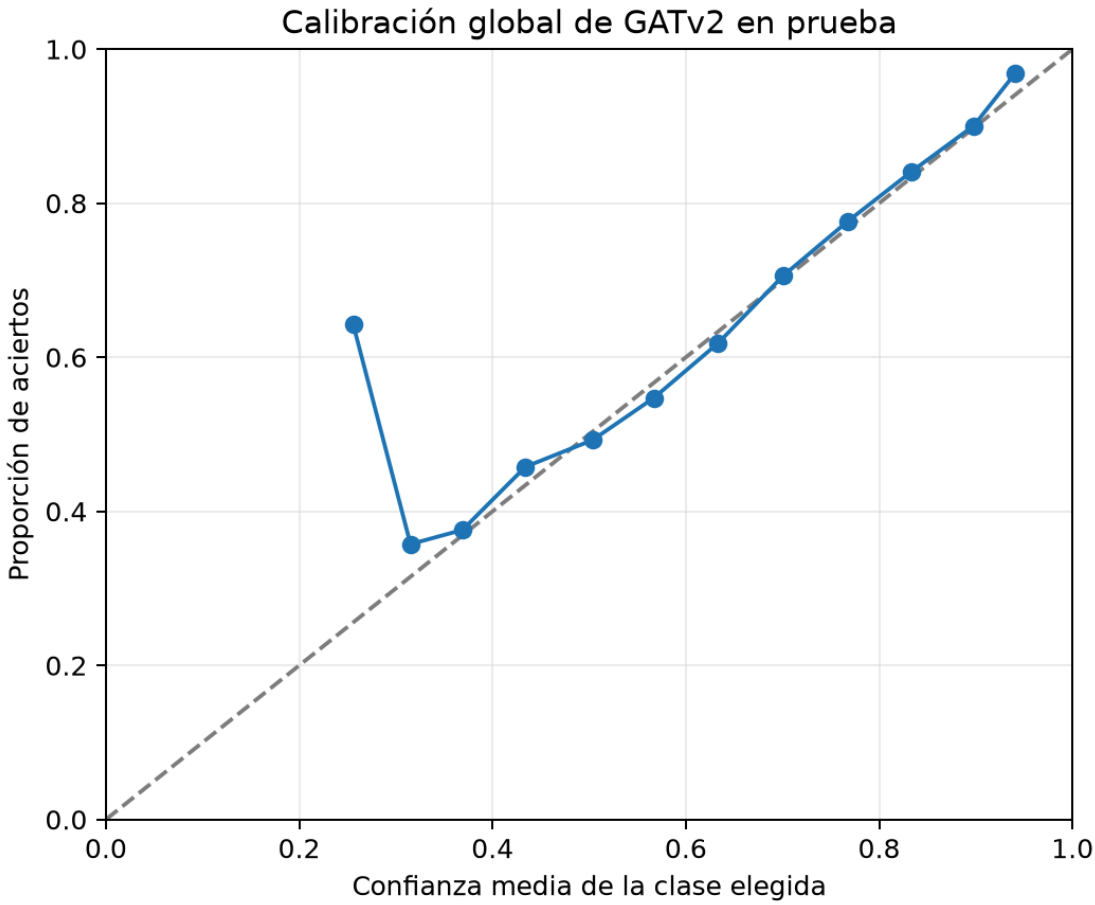
Esta prueba cuantifica la información sobre eventos que puede recuperar un clasificador simple del vector aprendido. Como el codificador recibió supervisión de esos eventos, el resultado no prueba descubrimiento autónomo de conceptos tácticos ni similitud semántica entre jugadas. La proyección PCA siguiente es descriptiva, ajustada sobre los vectores de prueba únicamente para visualizar; no interviene en entrenamiento, selección ni métricas predictivas.

Proyección descriptiva de representaciones de GATv2



Nota. Elaboración propia; partición 42, seis partidos de prueba.

Calibración global de la clase elegida



Nota. Elaboración propia; partición 42, seis partidos de prueba.

Predicciones y límites de esta primera ejecución

Se exportaron predicciones para los seis partidos de prueba con frame, identificador del portador, diez probabilidades de evento y cinco candidatos a receptor con sus probabilidades condicionales. Los archivos mantienen las columnas y metadatos que espera el visor. Se guardaron por separado, sin reemplazar predicciones existentes en Datos.

Esta evaluación utiliza una partición y no reemplaza la repetición en semillas 7 y 123. Conserva el pipeline original: suavizado centrado de 0.2 segundos y diferencias centrales para calcular cinemática, que utilizan observaciones posteriores. Además, el etiquetador admite eventos cuyo inicio coincide con el frame actual. La comparación es offline; una evaluación de anticipación estrictamente en línea requiere características causales, fijar un margen temporal y repetir todos los modelos. Las etiquetas heurísticas, especialmente gol, fuera de juego y falta, requieren validación con eventos oficiales.

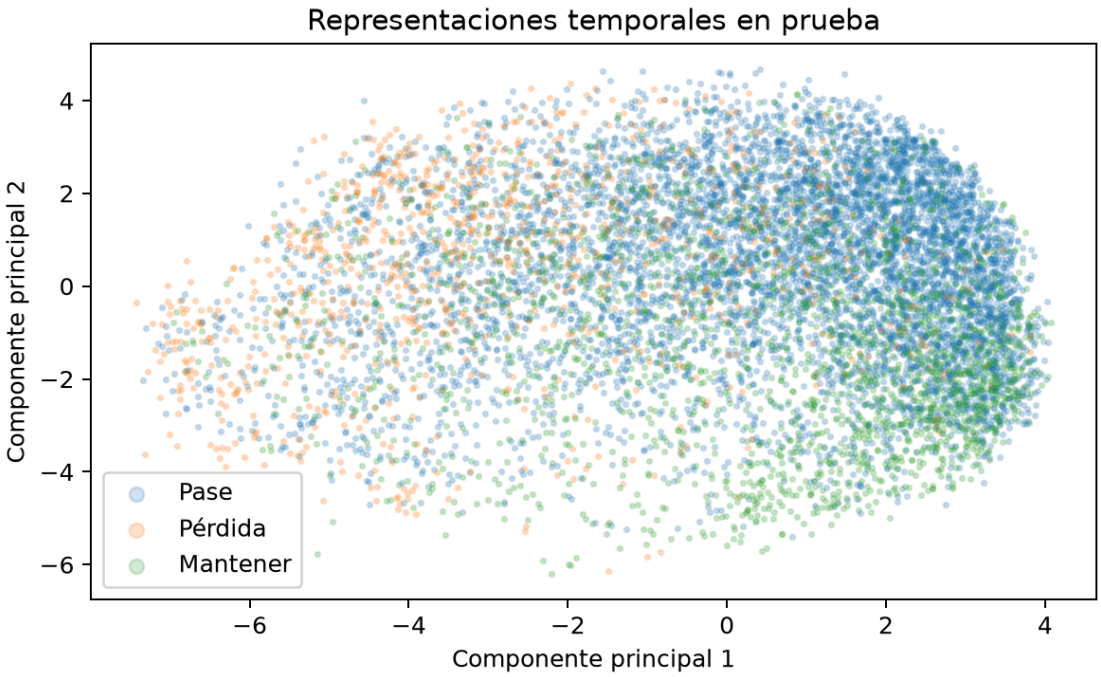
Representaciones temporales de la GNN con GRU

Representación vectorial del fragmento reciente

Exportamos vectores temporales de 64 componentes y evaluamos una prueba lineal con el sistema congelado. La regresión logística utilizó 30,000 ejemplos de entrenamiento, 10,000 de validación para seleccionar C entre 0.1 y 1, y 10,000 de prueba de partidos separados. Obtuvo log-loss 0.8255, accuracy 0.6880 y macro-F1 fiable 0.2422; la referencia por frecuencias en esa misma submuestra obtuvo log-loss 1.0338.

La prueba muestra que un clasificador simple recupera información sobre eventos de la representación supervisada. No valida similitud semántica entre jugadas ni conceptos tácticos descubiertos sin etiquetas. El vector resume una historia corta con cortes de continuidad, no una jugada completa de duración arbitraria. La PCA siguiente se ajustó sobre vectores de prueba únicamente para descripción visual; no intervino en entrenamiento ni selección.

Proyección descriptiva de las representaciones temporales



Nota. Elaboración propia; partición 42. La proyección PCA es descriptiva.

Se guardaron el codificador congelado, los pesos temporales, las divisiones, los historiales, el control T=1, las matrices y las predicciones de los seis partidos de prueba. Los Parquet mantienen las columnas del visor y expresan probabilidades de receptor condicionadas a un pase. Repetir otras particiones, validar eventos oficiales y evaluar características causales son pasos necesarios antes de generalizar las conclusiones a uso en línea.

Matriz de confusión

Las matrices de confusión permiten analizar los aciertos y errores de clasificación de eventos. Las filas representan las etiquetas de referencia y las columnas las predicciones de los modelos. Los soportes corresponden a observaciones de tracking y no necesariamente a jugadas independientes. GraphSAGE conserva la matriz agregada de tres particiones, mientras que Gradient Boosting, GATv2 y GNN con GRU presentan resultados de la semilla 42 sobre seis partidos de prueba. Los Graph Transformers se analizan posteriormente mediante métricas de clasificación obtenidas sobre cuatro partidos de prueba, por lo que sus resultados no deben compararse directamente con los del protocolo original.

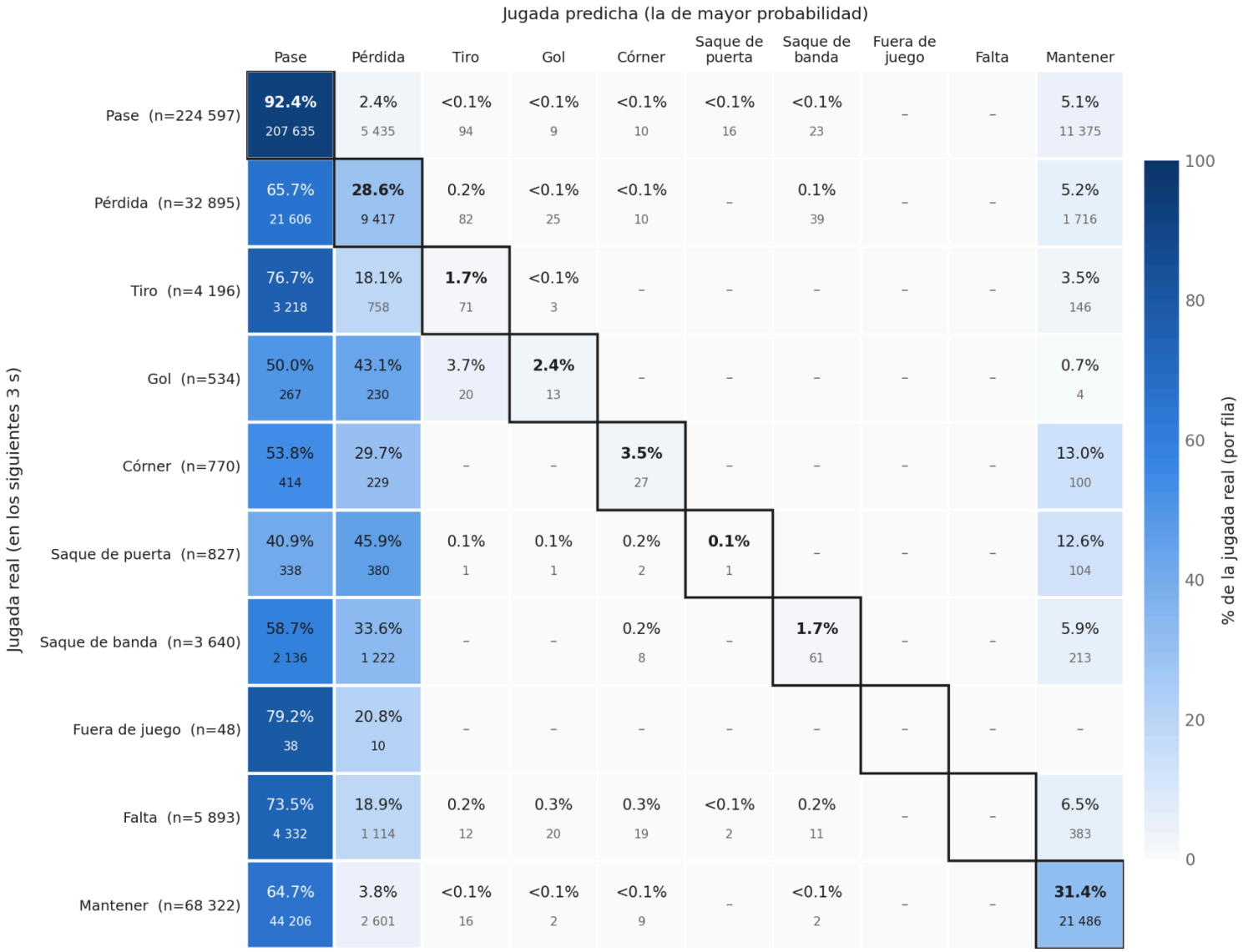
Árboles de decisión con Gradient Boosting

Matriz de conteos en los seis partidos de prueba de la semilla 42. Se presenta como tabla para conservar sin cambios las imágenes originales del documento.

Real / pred.	1	2	3	4	5	6	7	8	9	10
1	72610	1926	77	23	10	23	86	2	12	3766
2	7345	3190	70	12	8	14	43	1	16	547
3	1141	254	85	6	0	1	0	0	0	58
4	69	77	8	8	0	0	0	0	0	2
5	165	81	2	0	11	3	0	1	0	28
6	125	94	7	0	3	4	3	0	0	13
7	656	315	3	0	0	0	76	0	2	35
8	3	1	0	0	0	0	0	0	0	0
9	1540	463	6	0	9	2	1	0	4	88
10	16382	847	43	1	5	9	27	0	19	6219

Clave de clases: 1 Pase; 2 Pérdida; 3 Tiro; 4 Gol; 5 Córner; 6 Saque de puerta; 7 Saque de banda; 8 Fuera de juego; 9 Falta; 10 Mantener. Nota. Elaboración propia a partir de metrics.json de gbt_seed42_reproducido; N=118,786.
El recall del pase fue 92.5% y su precisión 72.6%. La accuracy total fue 69.2%. Estas medidas no equivalen a la probabilidad individual asignada a un pase ni al ECE; el comportamiento fuera de la diagonal permite identificar confusiones entre clases.

GraphSAGE



Cada celda: porcentaje de la fila y número de muestras. Recuadro oscuro = predicción correcta.

Matriz de confusión de GraphSAGE
Filas: evento de referencia obtenido mediante las reglas heurísticas de etiquetado.
Columnas: la que predijo el modelo (la de mayor probabilidad).
Cada celda muestra el porcentaje de su fila y, debajo, el número de muestras.
Usamos porcentajes porque con los conteos de los pases se taparía todo lo demás.
La diagonal con recuadro oscuro son los aciertos. En pases, 92.4% (207,635 de 224,597).

Interpretación de la matriz

- Los pases se reconocen muy bien. El 92.4% de los pases reales se clasifican como pase, y la mayoría de los fallos son "mantener el balón". Tiene sentido: el jugador podía pasar o seguir con el balón, y la diferencia es sutil.
- El modelo predice "pase" de más. Solo el 73% de sus predicciones de pase son correctas. El mayor error es en "mantener el balón": 44,206 casos, el 64.7% de esa clase, se predicen como pase. Le siguen las pérdidas (21,606, el 65.7%).
- Esto se debe a que dos de cada tres muestras son pases. Cuando el modelo duda, "pase" es la apuesta más segura.
- Las jugadas raras casi nunca se predicen, y sus errores caen sobre todo en pase o pérdida. Por ejemplo, 76.7% de los tiros se clasifican como pases. Fuera de juego y falta tienen 0 aciertos.
- Ambos modelos son prácticamente iguales en pases (92.4% frente a 92.8%). La ventaja de GraphSAGE está en la calidad de las probabilidades (log-loss), no en este conteo de aciertos.

Se sumaron las matrices de los 3 repartos de test: 341,722 muestras por modelo, de las que 224,597 son pases reales. La predicción de cada muestra es la jugada con mayor probabilidad. Una misma observación puede aparecer en más de una partición; el agregado no equivale a 341,722 ejemplos independientes únicos.

	GraphSAGE	**Gradient boosting**
Pases reales	224,597	224,597
Pases clasificados correctamente como pase	207,635 (92.4%)	208,372 (92.8%)
Pases que el modelo creyó "posesión"	11,375 (5.1%)	10,395 (4.6%)
Pases que el modelo creyó "pérdida"	5,435 (2.4%)	5,209 (2.3%)
Veces que el modelo predijo "pase"	284,190	287,967
De esas, eran realmente pase (precisión)	73.1%	72.4%

GATv2 relacional

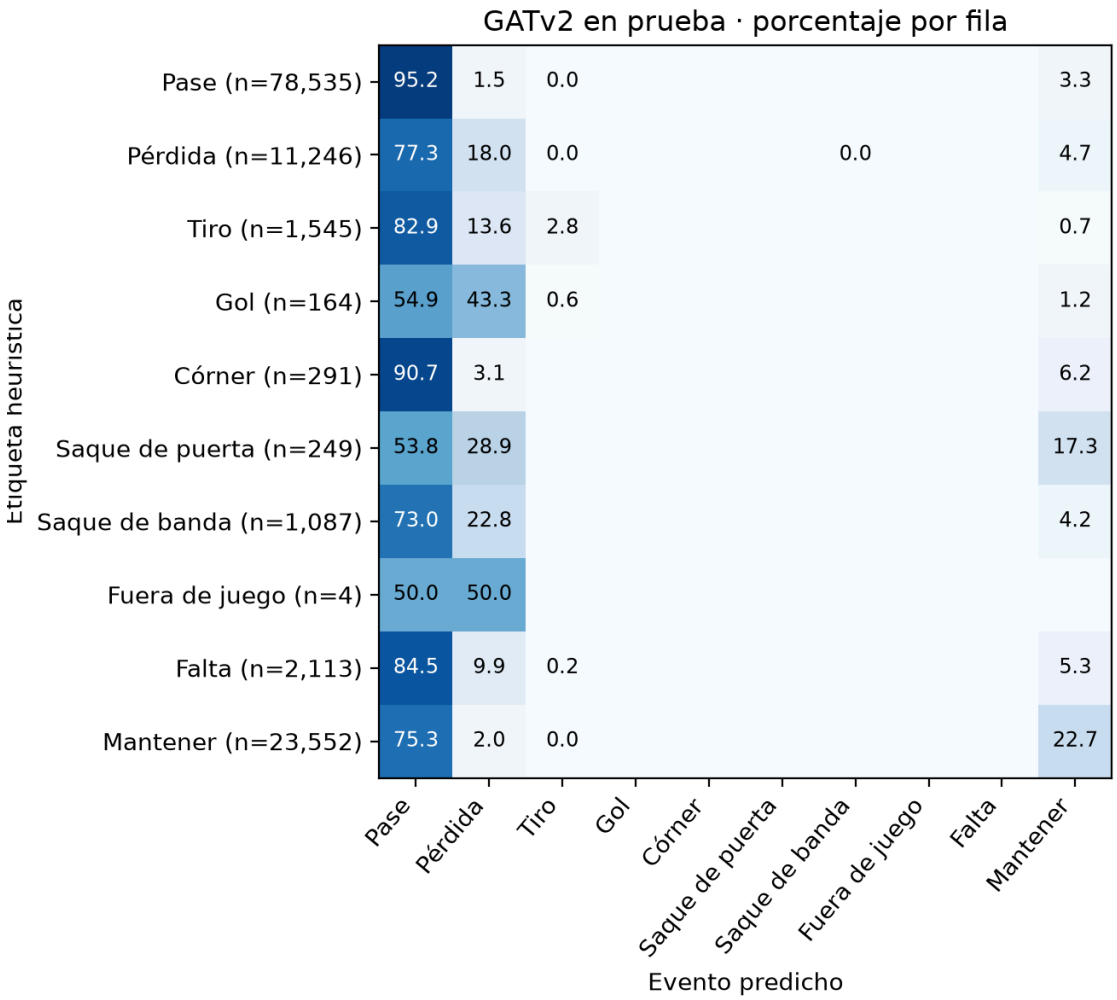
Clase	**Soporte**	**Precisión**	**Recall**	**F1**
Pase	78,535	0.708	0.952	0.812
Pérdida	11,246	0.449	0.180	0.257
Tiro	1,545	0.672	0.028	0.053
Gol	164	0.000	0.000	0.000
Córner	291	0.000	0.000	0.000
Saque de puerta	249	0.000	0.000	0.000
Saque de banda	1,087	0.000	0.000	0.000
Fuera de juego	4	0.000	0.000	0.000

Clase	Soporte	Precisión	Recall	F1
Falta	2,113	0.000	0.000	0.000
Mantener	23,552	0.616	0.227	0.331

El soporte cuenta muestras de tracking cuya primera acción futura tiene esa etiqueta, no jugadas distintas. Una misma acción puede etiquetar varios instantes de los tres segundos anteriores; por tanto, estos conteos no equivalen a un número independiente de pases, tiros o faltas.

El recall de pase fue 95.2 %: esa proporción de las etiquetas de pase se recuperó mediante argmax. La precisión fue 70.8 %: esa proporción de las predicciones de pase coincidió con la referencia heurística. Son cantidades distintas de una probabilidad individual o del ECE. El soporte y F1 por clase permiten revisar qué resultados siguen sin detectarse pese a un buen promedio global.

Matriz de confusión de GATv2 en prueba



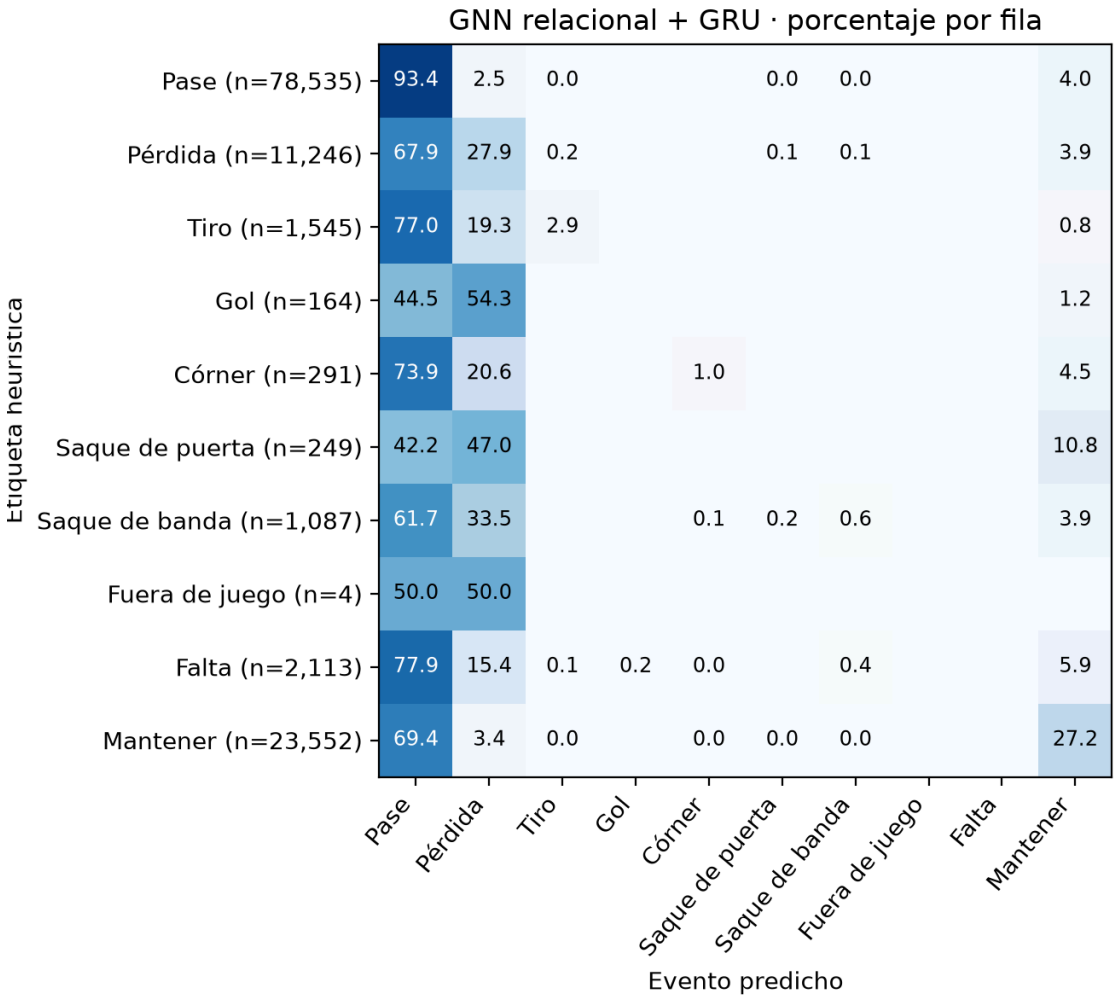
Nota. Elaboración propia; partición 42, seis partidos de prueba.

GNN relacional con GRU temporal

Clase	Soporte	Precisión	Recall	F1
Pase	78,535	0.725	0.934	0.816
Pérdida	11,246	0.439	0.279	0.341
Tiro	1,545	0.484	0.029	0.055
Gol	164	0.000	0.000	0.000
Córner	291	0.375	0.010	0.020
Saque de puerta	249	0.000	0.000	0.000
Saque de banda	1,087	0.175	0.006	0.012
Fuera de juego	4	0.000	0.000	0.000
Falta	2,113	0.000	0.000	0.000
Mantener	23,552	0.625	0.272	0.379

El soporte cuenta muestras etiquetadas, no jugadas independientes. Las clases gol, saque de puerta, fuera de juego y falta mantuvieron F1 igual a cero. Tiro, córner y saque de banda tuvieron recall bajo. La información temporal no resolvió el desequilibrio ni la incertidumbre del etiquetado heurístico; las mejoras agregadas no deben ocultar estas limitaciones.

Matriz de confusión del modelo GNN relacional con GRU



Nota. Elaboración propia; entrenamiento y evaluación en partidos separados.

Comparación de los modelos Graph Transformer

Para comparar las arquitecturas espaciales y temporales se utilizaron las mismas observaciones del conjunto de prueba, compuesto por cuatro partidos.

La comparación se realizó sobre 50,042 observaciones compatibles con la construcción de secuencias temporales.

Métrica	Graph Transformer espacial	Graph Transformer temporal	Temporal ponderado

Accuracy de eventos	65.59 %	70.66 %	66.45 %
Macro-F1	No calculado	0.1545	0.2012
Balanced accuracy	No calculado	16.44 %	26.56 %
Receptor top-1	48.04 %	33.47 %	31.50 %

Incorporación de información temporal

El Graph Transformer temporal alcanzó una precisión de 70.66 % en la predicción de eventos, frente al 65.59 % obtenido por el modelo espacial sobre las mismas observaciones.

Sin embargo, la clase mayoritaria representó el 69.55 % de las observaciones de prueba. Por lo tanto, el resultado temporal supera esta referencia por aproximadamente 1.11 puntos porcentuales.

Esto demuestra que la precisión global, por sí sola, no permite determinar si el modelo reconoce adecuadamente todas las categorías.

Predicción del receptor

El Graph Transformer espacial obtuvo un resultado superior en la identificación del receptor del balón, con una precisión top-1 de 48.04 %.

En contraste, los modelos temporales alcanzaron 33.47 % y 31.50 %, respectivamente.

Estos resultados indican que la incorporación de información temporal no produjo una mejora uniforme en los dos objetivos de predicción. La diferencia también puede estar relacionada con las características utilizadas y el entrenamiento de los cabezales, por lo que no debe atribuirse exclusivamente al mecanismo de atención temporal.

Efecto de la ponderación de clases

El modelo temporal ponderado obtuvo una precisión global de 66.45 %, inferior al 70.66 % del modelo temporal original.

Sin embargo, presentó mejoras en las métricas que consideran el desempeño de las categorías minoritarias.

El macro-F1 aumentó de 0.1545 a 0.2012, equivalente a una mejora relativa aproximada del 30.2 %. Por su parte, el balanced accuracy aumentó de 16.44 % a 26.56 %.

Estas mejoras muestran que la ponderación favoreció una distribución más equilibrada de las predicciones entre las clases.

Particularmente, la clase 2 aumentó su F1 de 0.09 a 0.24, mientras que la clase 6 incrementó su recall de 7 % a 50 %.

No obstante, algunas categorías continuaron presentando un F1 igual a cero, lo que evidencia que la ponderación no resuelve completamente las dificultades asociadas al desbalance y a la escasez de ejemplos.

Alcance de los resultados

Los experimentos Graph Transformer utilizaron una selección de 10 partidos de entrenamiento, dos de validación y cuatro de prueba, mientras que las ejecuciones principales de los otros modelos utilizaron una partición de 46, seis y seis partidos.

Por esta razón, los resultados de los Graph Transformer se presentan como evidencia exploratoria adicional y no como una comparación directa de superioridad frente a GraphSAGE, GATv2 o la GNN con GRU.

Se requieren experimentos adicionales con particiones idénticas y un mayor número de épocas para determinar con mayor precisión el efecto de las diferencias arquitectónicas.

Conclusión y selección del modelo final

En este avance se desarrollaron y evaluaron seis arquitecturas para abordar la predicción del siguiente evento futbolístico y del posible receptor del balón. Los modelos implementados fueron Gradient Boosting, GraphSAGE, GATv2 relacional, GNN relacional con GRU temporal, Graph Transformer espacial y Graph Transformer espacial-temporal. Adicionalmente, se evaluó una variante del último modelo con ponderación de clases, con el objetivo de mejorar el reconocimiento de eventos poco frecuentes.

Los resultados muestran que cada arquitectura presenta ventajas y limitaciones. Gradient Boosting constituye una referencia tabular; GraphSAGE y GATv2 permiten aprender representaciones espaciales de las relaciones entre jugadores; la GNN con GRU incorpora información histórica; y los Graph Transformers exploran mecanismos de atención espacial y temporal. Sin embargo, una mayor complejidad arquitectónica no garantiza necesariamente un mejor desempeño en todas las métricas.

Comparación del desempeño de los modelos

En las tres particiones originales, GraphSAGE obtuvo un log-loss promedio de 0.803, frente a 0.826 de Gradient Boosting y una referencia de 1.036 basada en las frecuencias de las clases. Esta mejora indica una mayor capacidad para asignar probabilidades a los eventos observados, aunque no necesariamente se traduce en mejores resultados de calibración o macro-F1.

Por su parte, Gradient Boosting mantuvo un desempeño competitivo y alcanzó un mayor macro-F1 fiable bajo decisión argmax que GATv2 y GRU en la semilla 42, lo que demuestra la importancia de conservar modelos tradicionales como referencia.

En los experimentos correspondientes a la semilla 42, GATv2 alcanzó un log-loss de eventos de 0.8202, frente a 0.8304 de Gradient Boosting. La GNN relacional con GRU temporal redujo esta pérdida a 0.8082 y obtuvo una precisión top-1 de receptor de 51.20 %, superando el 49.89 % de GATv2 y el 50.64 % de Gradient Boosting. No obstante, GATv2 presentó un menor error de calibración esperado (ECE), por lo que ninguna arquitectura dominó todas las métricas.

Como ampliación experimental, se implementaron dos arquitecturas basadas en Graph Transformers. El modelo espacial alcanzó una precisión de 65.59 % en eventos y 48.04 % en la identificación del receptor. Al incorporar un Transformer temporal, la precisión de eventos aumentó a 70.66 %, mientras que la precisión del receptor disminuyó a 33.47 %.

Aunque esta mejora de 5.07 puntos porcentuales en eventos sugiere que la información temporal puede resultar útil, el análisis del desbalance mostró que la clase mayoritaria representaba el 69.55 % de las observaciones de prueba. Por tanto, el modelo temporal superó esta referencia por apenas 1.11 puntos porcentuales.

Para atender esta limitación, se desarrolló una variante temporal con ponderación de clases. Aunque su precisión global disminuyó a 66.45 %, el macro-F1 aumentó de 0.1545 a 0.2012 y el balanced accuracy pasó de 16.44 % a 26.56 %. Estos resultados sugieren que la ponderación favoreció el reconocimiento de algunas clases minoritarias, aunque persistieron dificultades para identificar categorías con pocos ejemplos.

Es importante señalar que los Graph Transformers se entrenaron utilizando diez partidos para entrenamiento, dos para validación y cuatro para prueba, debido a las limitaciones de recursos computacionales y al tiempo disponible. En contraste, los primeros cuatro

modelos utilizaron principalmente una partición de 46 partidos de entrenamiento, seis de validación y seis de prueba. Por esta razón, los resultados de ambos grupos no permiten establecer comparaciones directas de superioridad entre arquitecturas.

Selección de los dos mejores modelos

Considerando los resultados obtenidos, seleccionamos **GraphSAGE y la GNN relacional con GRU temporal como los dos modelos finalistas** para continuar el desarrollo del proyecto.

La selección de GraphSAGE se fundamenta en la consistencia de sus resultados a través de tres particiones independientes. Su log-loss promedio de 0.803 fue inferior al de Gradient Boosting, lo que representa una ventaja en la calidad de las probabilidades estimadas. Además, su capacidad para aprender relaciones espaciales entre jugadores constituye una base relevante para representar el comportamiento colectivo durante los partidos.

Por otro lado, seleccionamos la GNN relacional con GRU temporal debido a su desempeño competitivo en la semilla 42 y a su capacidad para incorporar información histórica. Su log-loss de 0.8082 y su precisión top-1 de receptor de 51.20 % la posicionaron como una alternativa prometedora dentro de las arquitecturas evaluadas bajo ese protocolo.

Esta selección considera tanto el desempeño cuantitativo como la pertinencia de las arquitecturas para el objetivo del proyecto. No obstante, reconocemos que el promedio de GraphSAGE y los resultados de GRU en la semilla 42 no constituyen una comparación completamente emparejada, por lo que la selección debe entenderse como una decisión fundamentada en la evidencia disponible y no como una demostración de superioridad estadística.

Ajuste y evaluación de los modelos seleccionados

El desarrollo experimental permitió explorar distintas configuraciones y estrategias para mejorar el desempeño de las arquitecturas.

En GraphSAGE se evaluó el comportamiento del modelo en tres particiones de datos, utilizando los resultados de validación y prueba para analizar su estabilidad y capacidad de generalización.

En la GNN relacional con GRU temporal se incorporó una secuencia de observaciones anteriores y se realizó un experimento de control con una única observación (T=1), con el propósito de identificar la contribución de la información temporal. Los resultados mostraron una mejora limitada al incorporar historia, lo que sugiere que la información temporal puede aportar valor, aunque no explica por sí sola todas las diferencias observadas.

Adicionalmente, los experimentos con Graph Transformers permitieron explorar el efecto de modificar la arquitectura y la función de pérdida. Particularmente, la ponderación de clases mostró mejoras en macro-F1 y balanced accuracy, lo que representa una línea de trabajo relevante para futuras optimizaciones de los modelos seleccionados.

Sin embargo, los ajustes realizados no constituyen una búsqueda exhaustiva de hiperparámetros ni una optimización equivalente de los dos finalistas. Para confirmar cuál configuración es óptima sería necesario realizar una búsqueda sistemática utilizando las mismas particiones y criterios de selección.

Elección del modelo individual final

A partir de los resultados disponibles, **seleccionamos la GNN relacional con GRU temporal como el modelo individual final para continuar el proyecto**.

Esta decisión se fundamenta en tres consideraciones principales.

En primer lugar, el modelo presentó un desempeño competitivo en ambos objetivos de predicción. En la semilla 42 obtuvo un log-loss de eventos de 0.8082 y una precisión top-1 de receptor de 51.20 %, superando en ambas métricas a Gradient Boosting y GATv2 bajo la misma partición.

En segundo lugar, su arquitectura permite integrar las relaciones espaciales entre jugadores con información de observaciones anteriores. Esta característica resulta especialmente relevante para el objetivo general del proyecto, que busca avanzar desde la descripción de posiciones individuales hacia la comprensión de comportamientos colectivos y la anticipación del desarrollo de las jugadas.

En tercer lugar, el modelo ofrece una base para futuras extensiones, como la incorporación de ventanas temporales más amplias, nuevas características tácticas y mecanismos de atención que permitan identificar interacciones relevantes entre jugadores.

La elección de la GNN con GRU no implica que haya demostrado ser universalmente superior a GraphSAGE o a las demás arquitecturas. Se trata de una selección práctica basada en su desempeño bajo el protocolo experimental disponible y en su alineación con los objetivos de investigación.

Limitaciones y trabajo futuro

Los resultados obtenidos deben interpretarse considerando algunas limitaciones metodológicas.

En primer lugar, existe un desbalance considerable entre las categorías de eventos. Los pases presentan generalmente el mayor recall, mientras que las pérdidas, retenciones y otros eventos poco frecuentes continúan representando un desafío.

En segundo lugar, la calidad de las etiquetas y la cantidad de eventos independientes condicionan la interpretación de las métricas. Asimismo, aunque el ECE global de las evaluaciones originales fue bajo, esta medida no garantiza una calibración adecuada para cada categoría individual, por lo que sería necesario realizar análisis específicos por clase (Guo et al., 2017).

En tercer lugar, las diferencias entre los protocolos experimentales y las limitaciones computacionales impiden comparar directamente todas las arquitecturas bajo condiciones equivalentes.

Como trabajo futuro, consideramos prioritario realizar una evaluación emparejada de GraphSAGE y GNN con GRU, ampliar la búsqueda de hiperparámetros y explorar estrategias de ponderación de clases. También sería relevante entrenar los Graph Transformers con el conjunto completo de partidos y desarrollar arquitecturas espaciales-temporales que permitan optimizar conjuntamente sus componentes.

En conclusión, **la GNN relacional con GRU temporal se selecciona como el modelo individual final de este avance**, mientras que GraphSAGE permanece como el principal modelo alternativo de referencia. Esta decisión permite establecer una dirección concreta para las siguientes etapas del proyecto, sin perder de vista que la predicción de eventos futbolísticos requiere equilibrar la calidad de las representaciones espaciales y temporales, la calibración de las probabilidades y la capacidad para reconocer eventos minoritarios.

Referencias

Brody, S., Alon, U., & Yahav, E. (2022). *How attentive are graph attention networks?* International Conference on Learning Representations. https://arxiv.org/abs/2105.14491

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. En *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP) (pp*. 1724–1734). Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1179

García Austria, H. (s. f.). *Soccer play prediction (rama new-graphsage-model)* [Código fuente, repositorio privado]. GitHub. https://github.com/Hillelman/soccer-play-prediction

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. En *Proceedings of the 34th International Conference on Machine Learning (Vol*. 70, pp. 1321–1330). PMLR. https://proceedings.mlr.press/v70/guo17a.html

Hamilton, W. L., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. En *Advances in Neural Information Processing Systems (Vol*. 30). https://papers.nips.cc/paper_files/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html

PyTorch contributors. (s. f.). *GRU.* PyTorch documentation. Recuperado el 25 de octubre de 2026, de https://docs.pytorch.org/docs/2.14/generated/torch.nn.GRU.html

PyTorch Geometric contributors. (s. f.). *Heterogeneous graph learning.* PyTorch Geometric documentation. Recuperado el 25 de octubre de 2026, de https://pytorch-geometric.readthedocs.io/en/latest/notes/heterogeneous.html

scikit-learn developers. (s. f.). *HistGradientBoostingClassifier.* scikit-learn documentation. Recuperado el 25 de octubre de 2026, de https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html